Wave Forecaster: o que já foi portado para a GPU

Versão curta do relatório para o Prof. Pedro Cruz (GTA, DEL/Poli/UFRJ), em português técnico controlado

Matheus Hoffmann, Escola Politécnica da UFRJ
2026-10-20

> **Esta é a versão curta da terceira versão do relatório sobre o projeto ww3-gpu.** A leitura leva cerca de 15 minutos. A versão completa tem as figuras, as tabelas de ADRs e WFIPs e a comparação com o ww3-dia-gpu.
>
> - **Vale o original.** Se este texto e os arquivos do repositório discordam, valem os arquivos: `kokkos/PORT_STATUS.md`, `kokkos/ports.tsv` e o `README.pt-BR.md`.
> - **Feita a partir de:** o repositório em 2026-10-20, commit `b8a6463` de `main`.
> - **Como foi escrita:** o texto foi gerado por Claude com a skill Português Técnico Controlado (PTC), versão 1.1.0, de Kayque Rotondo, no nível `descritivo`. A revisão é de Matheus Hoffmann.

Marcas. (v) quer dizer "verificado numa fonte ou num comando", e o texto nomeia essa fonte. ⚠ quer dizer "ainda não verificado".

---

1. O problema

O modelo. O WAVEWATCH III® (WW3) é o modelo de previsão de ondas da NOAA, a agência americana de oceano e atmosfera. O WW3 é um programa em Fortran com 263.375 linhas (v, `kokkos/PORT_STATUS.md`).

O que o WW3 calcula. O modelo divide o mar numa grade de pontos. Em cada ponto de mar, o WW3 guarda um espectro: um vetor que diz quanta energia de onda existe em cada frequência e em cada direção. A cada passo de tempo, o programa faz duas etapas:

1. **Propagação:** o programa move a energia de um ponto para os pontos vizinhos, como um *stencil* sobre uma imagem.
2. **Termos-fonte:** em cada ponto, o programa soma a energia que o vento coloca, a energia que a quebra das ondas tira e a energia que as ondas trocam entre si. Nesta etapa, um ponto não depende de outro ponto.

Por que a GPU ajuda. A segunda etapa é um *map* sobre um vetor grande de pontos independentes (9.520 pontos de mar no caso `full`). Uma GPU executa esse formato bem. No perfil medido, só a troca de energia entre ondas (o DIA, *Discrete Interaction Approximation*) ocupa 48,4 % do tempo.

A regra que torna o problema difícil. A versão em GPU precisa dar **o mesmo resultado, bit a bit**, que o Fortran original, dentro do mesmo executável do WW3. Se um bit muda, a tradução está errada até que duas execuções do próprio Fortran mostrem a mesma diferença.

---

2. A fase atual do projeto

Velocidade primeiro, validação depois. Em 2026-10-17, o dono do projeto fixou o objetivo principal: deixar o WW3 mais rápido com Kokkos e CUDA, sem mudar a aritmética do Fortran (ADR-0016; ADR quer dizer *Architecture Decision Record*). A mesma decisão adiou os regtests do WW3 para uma **fase de validação**. O dono do projeto decide quando essa fase começa. Até lá, o projeto mostra a paridade por análise estática do código que os dois compiladores produzem com `-O3`. Todo número de velocidade desta fase leva a marca "regtest pendente ⚠".

O que mudou de 2026-10-16 a 2026-10-19. O `main` recebeu 269 commits em 39 pull requests, do #158 ao #206 (v, `git log 3445100..b8a6463`):

1. **Tradução em massa** (PRs de #159 a #168): as rotinas traduzidas foram de 91 para 316, e as rotinas ligadas ao WW3 foram de 10 para 72.
2. **O passo de tempo residente** (PRs #173, #179, #184 e #191): com `--kokkos resident`, o espectro fica no lado do Kokkos durante a execução inteira, e sete partes do passo de tempo executam em Kokkos.
3. **A matemática da GPU** (PRs #177 e #178): as chamadas a `expf` e `logf` usam cópias das funções da `glibc` compiladas para a GPU, porque a biblioteca da NVIDIA usa outros algoritmos. Nenhuma execução na GPU comparou ainda essas cópias.
4. **Produção num nó só** (PR #191): o modo estrito interrompe a execução quando uma parte volta para o Fortran sem aviso. O `ww3_multi` e os builds com OpenMP deram saídas idênticas byte a byte às do Fortran (ADR-0017).
5. **Pontos em vetor e grade não estruturada** (PRs #199, #201, #202 e #204): o código Kokkos calcula oito pontos ao mesmo tempo, um por posição do registrador SIMD da CPU, e a propagação na grade não estruturada (`W3XYPUG`) deu os bytes do Fortran no regtest `ww3_tp2.6`.
6. **O caminho para o GFS-Wave** (PR #206): o GFS-Wave é o sistema que a NOAA usa na previsão operacional de ondas. O `W3SRCE` em lote foi para a switch dos regtests do GFS-Wave (plano 007).

---

3. Como o código é organizado

O ww3-gpu não edita o WW3. O código do WW3 vem de submódulos do git, e o ww3-gpu acrescenta só a reescrita em C++ e as ferramentas em volta (v, `README.pt-BR.md`, "como funciona"):

- `WW3-noaa/` é o WW3 da NOAA no commit `03adda91`, sem mudança. É a referência.
- `WW3/` é o fork h0ffmann/WW3 no commit `96892c95`: o WW3 da NOAA mais o patch que chama o código novo, atrás da chave de compilação `KOKKOS`. O patch muda 9 arquivos e acrescenta 7, com 5.902 linhas

acrescentadas e nenhuma removida (v, `git -C WW3 diff --shortstat 03adda91 96892c95 -- model`).

Kokkos é uma biblioteca C++ do Sandia National Laboratories. Quem programa escreve o kernel uma vez e compila esse kernel para Serial, OpenMP (vários núcleos da CPU) ou CUDA (GPU NVIDIA). O Fortran chama o C++ pela interface `bind(C)` do Fortran.

Um executável, dois caminhos. O executável contém as duas versões de cada rotina ligada. A variável de ambiente `WW_KOKKOS_<PARTE>` escolhe o Fortran (`0`) ou o Kokkos (`1`), sem recompilar. Assim, o teste compara as duas versões com o mesmo binário, a mesma entrada e o mesmo compilador.

Por que o espectro fica no lado do Kokkos. A GPU tem memória própria. No benchmark do DIA, a chamada inteira levou 0,75 ms e o kernel levou 0,047 ms: as cópias de memória foram cerca de 94 % da chamada (v, `bench/results/`). O preset `resident` copia o espectro uma vez, no começo, e devolve o espectro ao Fortran só quando o WW3 escreve uma saída ou um restart.

---

4. O que já foi portado

O que "portado" quer dizer. Uma rotina está **portada** quando o WW3 chama o código Kokkos e esse código passa nos testes de regressão originais da NOAA (os *regtests*) com o resultado do Fortran. O ww3-gpu não muda a lógica dos regtests: os regtests são a tabela verdade.

Nível	O que quer dizer	Parte do tempo	Rotinas	Linhas do WW3
traduzido	o código Kokkos existe e foi conferido só por leitura	**99,3 %**	316 de 1.023	69.024 (26,21 %)
ligado	o WW3 do fork chama esse código, atrás da chave	**99,3 %**	72 de 1.023	19.059 (7,24 %)
portado	os regtests deram o resultado do Fortran com esse código dentro do WW3	**48,6 %**	14 de 1.023	3.151 (1,20 %)

v. `python3 scripts/port_score.py --check`, executado em 2026-10-20 no commit `b8a6463`. O código Kokkos tem 62.257 linhas de C++ em 355 arquivos. A distância entre 99,3 % e 48,6 % é o trabalho da fase de validação.

As 14 rotinas portadas (v, coluna `regtested` de `kokkos/ports.tsv`):

- **O DIA** (`W3SNL1` e `INSNL1`): idêntico byte a byte nas 125 entradas da matriz de regtests do WW3, 100 seriais e 25 com OpenMP (2026-10-15, `W3SNL1_KOKKOS.md`).

- **O suporte do `W3WAVE`** (`W3GATH`, `W3SCAT`, `W3UWND`, `W3DZXY`, `WAVNU2`, `DISTAB`, `W3MAP3` e `W3MAPT`): verificado nos casos `smoke` e `full`.
- **A grade não estruturada** (`W3XYPUG` e `W3XYPFSN2`): as duas entradas de `ww3_tp2.6`.
- **O `ww3_multi`** (`WMIOHG` e `WMIOEG`): as entradas de `mww3_test_01` a `mww3_test_05`.

Uma ressalva. O caso `smoke` é o regtest `ww3_ts1` da NOAA, mas o caso `full` é do próprio repositório. Oito das 14 rotinas passaram só pelos casos `smoke` e `full` e ainda não passaram pela matriz completa da NOAA.

---

5. Velocidade: o que foi medido

As medições usaram uma CPU Intel Core i9-14900 (24 núcleos, 32 threads) e uma GPU RTX 4090.

Medição	Resultado	Fonte
Modelo inteiro na CPU, caso `full`, 32 threads, preset `resident`	15,84 s contra 26,18 s: **1,65 vez mais rápido**; regtest pendente ⚠	mediana de dez, 2026-10-18, registro da NOAA, seção 3
O mesmo caso com 1 thread	174,78 s contra 174,27 s: o mesmo tempo	uma execução ⚠
Kernel do DIA, fora do WW3, CUDA na RTX 4090, com as cópias	0,75 ms contra 25,88 ms em 1 thread: 34 vezes	mediana de três, 2026-09-15, `bench/results/`
Modelo inteiro na GPU	nenhuma execução até 2026-10-20; o build CUDA compila	`cuda-build-only`

Como ler esses números. O ganho na CPU vem da divisão do trabalho entre threads: com 1 thread, o Kokkos e o Fortran levam o mesmo tempo. No build com MPI (*Message Passing Interface*), que o GFS-Wave usa, o WW3 executa o Fortran por decisão de projeto (ADR-0017). O valor de 34 vezes vale só para o kernel isolado, não para o WW3 inteiro.

---

6. Como a tradução é verificada

A tradução com disciplina. Cada instrução do Fortran vira uma instrução em C++, na mesma ordem, sem mudar a álgebra. O código Kokkos não tem testes próprios: os únicos testes são os regtests do WW3 (ADR-0010). A receita lembra a de Koldunov et al. (2026), que traduziram o modelo oceânico FESOM2 de Fortran para C++/Kokkos com um assistente de código (arXiv:2606.11356). A diferença: o ww3-gpu traduz direto, sem etapa em C, e testa dentro do WW3.

O regtest é a tabela verdade. Cada entrada da matriz `regtests/bin/matrix.base` executa duas vezes no mesmo executável, uma com o Fortran e uma com o Kokkos. O `test.comp` da NOAA compara as saídas, e um arquivo diferente reprova a rotina.

A ferramenta de tolerância: `nccmp-tol`. O `nccmp-tol` compara dois arquivos NetCDF variável por variável e mede o tamanho de cada diferença. Cada variável tem uma tolerância no arquivo `tolerances.txt`. Por exemplo, a altura de onda `hs` aceita até 0,0001 m de diferença absoluta ou 0,0001 de diferença relativa. Na CPU, o `nccmp-tol` nunca transforma uma diferença em aprovação: o veredicto é do `test.comp`, bit a bit. Na GPU, o `nccmp-tol` julga a execução pelas tolerâncias, sem alargar nenhuma tolerância (ADR-0014).

Por que bit a bit é difícil. Com a contração ligada, o compilador junta `a*b + c` numa instrução só (FMA, *fused multiply-add*), que arredonda uma vez e às vezes muda o último bit. Por isso o projeto compila com `-ffp-contract=off` e, na GPU, com `--fmad=false`. O gfortran também expande `x**11` de um jeito em `-O0` e de outro em `-O3` (v, `docs/log/`).

---

7. O próximo passo

Todas as rotinas de cálculo que o perfil mede já estão traduzidas. As três frentes abertas em 2026-10-20 são:

1. **O GFS-Wave.** O plano 007 leva o `W3SRCE` em lote e o `W3XYP3` para a switch dos regtests do GFS-Wave. O passo 1 está pronto, e os passos de 2 a 4 estão bloqueados.
2. **A primeira execução na GPU**, numa RTX 4090, supervisionada pelo dono do projeto (plano 003). Depois, a H100 do LabECO/UFSC: o pedido de acesso é a issue h0ffmann/ww3-gpu#143, aberta. Sem esse acesso, os números da H100 continuam projeções (⚠).
3. **As lacunas para uso operacional.** O `PRODUCTION_READINESS_202610.md` lista 13 lacunas. Três lacunas bloqueiam o uso no GFS-Wave: nenhuma switch de regtest tem os termos-fonte que o `W3SRCE` em Kokkos traduz; o MPI fica fora do código Kokkos; e a paridade não foi medida com o compilador da operação nem na GPU.

Depois dessas frentes, o dono do projeto vai iniciar a fase de validação.

Links e como citar

O quê	Onde
Código (GitHub)	https://github.com/h0ffmann/ww3-gpu
Resumo do projeto em português	`README.pt-BR.md`
Site	https://h0ffmann.github.io/ww3-gpu/pt-BR/
Versão completa deste relatório	https://h0ffmann.github.io/ww3-gpu/relatorio-gta/
Decisões (18 ADRs) e desenhos de mudança (5 WFIPs)	`docs/ADRs/` e `docs/WFIPs/`
A skill de escrita deste texto	Português Técnico Controlado (PTC), versão 1.1.0, de Kayque Rotondo, nível `descritivo`

O quê	Onde
Citação	o DOI 10.5281/zenodo.23221351 abre sempre a versão mais recente; a mais recente publicada é a `v0.1.3`, de 2026-10-09 (v, `RELEASES.md`)

Referências. `README.pt-BR.md` do ww3-gpu, commit `b8a6463`, lido em 2026-10-20, com números de 2026-10-17; `kokkos/PORT_STATUS.md`, commit `b8a6463`; `docs/NOAA_PROGRESS_RECORD_202610.md`, versão de 2026-10-18; Koldunov et al., 2026, arXiv:2606.11356; Rotondo, K., Português Técnico Controlado, versão 1.1.0, commit `cfdffad`, licença MIT.